

Transpilador Latim → Python

Equivalência Semântica entre Linguagem Fonte e Destino

Disciplina: Teoria da Computação e Compiladores

Instituição: UNIFACS — Universidade Salvador

Grupo: João Luccas Lordelo Marques - 12725224055

Maurício Gabriel leal da Silva - 12724157145

Pablo Ernesto da Cunha Guerreiro - 12724111921

Semestre: 2026.1

O Que É o Nosso Transpilador?



Conversão Fonte-a-Fonte

Um **transpilador** que converte código escrito em uma linguagem fictícia baseada em **Latim** para **Python** válido e executável.



Linguagem Fonte (Latim)

Projetada com **tipagem declarativa** e **pontuação rígida**, utilizando delimitadores como chaves { } e ponto final ..



Fases do Compilador

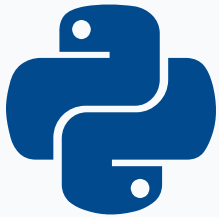
O projeto contempla todas as fases clássicas: análise **léxica, sintática, semântica e geração de código**.



Linguagem Destino (Python)

Exige **identação por espaços** e utiliza tipagem dinâmica por coerção, simplificando a estrutura visual.

Python: Clareza e Legibilidade como Princípio



- ✓ Python foi escolhido por sua **sintaxe limpa e legível**, facilitando a validação visual do código gerado.
- ✓ A **tipagem dinâmica** permite que o transpilador aplique coerção de tipos via `int()`, `float()` e `bool()` diretamente nos pontos de entrada.
- ✓ O código de saída gerado é **imediatamente executável** sem necessidade de compilação adicional.
- ✓ A escolha reforça o objetivo: demonstrar **equivalência semântica de forma transparente e auditável**.

Equivalência Semântica – Conceito e Aplicação

O CONCEITO

Equivalência semântica garante que dois programas, escritos em linguagens distintas, produzam **o mesmo comportamento observável** para qualquer entrada válida.

No nosso transpilador, cada construção da linguagem Latim possui um **mapeamento direto e determinístico** para uma construção Python equivalente.

A APLICAÇÃO

📊 A **Tabela de Símbolos** é consultada em tempo de transpilação para garantir que os tipos declarados na fonte sejam preservados na saída.

📦 O barema exige no mínimo **4 tipos de dados** — todos implementados e mapeados rigorosamente para garantir a equivalência.

Mapeamento de Tipos: Da Declaração à Coerção

Tipo em Latim	Equivalente Python	Comportamento
<code>numerus</code>	<code>int()</code>	Inteiro; gera cast automático no <code>input()</code>
<code>decimalis</code>	<code>float()</code>	Ponto flutuante; cast via <code>float(input())</code>
<code>textus</code>	<code>str</code>	String nativa; sem cast adicional
<code>logicum</code>	<code>bool</code>	Booleano; validação de expressões lógicas

- A **Tabela de Símbolos** armazena o tipo de cada variável declarada.
- No momento da leitura (`lege()`), o transpilador consulta a tabela e aplica o **cast correto automaticamente**, garantindo segurança de tipos.

lege() e scribe() – Comunicação com o Usuário

[➔] scribe(item).

➔ `print(item)`

Função de **saída** de dados no console;
mapeamento direto e sem ambiguidade.

lege(var).

➔ `input()` com `cast`

O transpilador substitui o token e **consulta a Tabela de Símbolos** para aplicar `int(input())` ou `float(input())` conforme necessário.



Esse comportamento demonstra que a equivalência não é apenas **lexical** (troca de palavras), mas **semântica** (preservação do comportamento e dos tipos).

Condicionais: Da Chave { } à Identação Python

- Em Latim, os blocos condicionais são delimitados por **chaves { }**, tornando a estrutura explícita e independente de espaçamento.
- O **parser** identifica as chaves como delimitadores de bloco e as **descarta**, convertendo o conteúdo interno para o padrão de **identação por 4 espaços** do Python.
- Esse processo é um exemplo claro de **transformação sintática com preservação semântica**: a estrutura de controle de fluxo é idêntica em ambas as linguagens.

LATIM

```
si (expr) {  
    ...  
}
```



PYTHON

```
if expr:  
    ...
```

LATIM

```
aliter {  
    ...  
}
```



PYTHON

```
else:  
    ...
```

Laços: dum e pro – Equivalência por Contagem e Condição

↻ Laço dum

Mapeamento **direto e semântico**: laço condicional contínuo, executado enquanto a expressão for verdadeira.

LATIM (FONTE)

```
dum (expr) { scribe(x). }
```

PYTHON (DESTINO)

```
while expr: print(x)
```

≡ Laço pro

O laço for clássico por contagem **não possui equivalente direto** em Python com a mesma semântica. O transpilador gera um while equivalente.

LATIM (FONTE)

```
pro (i=0; i<x; i=i+1) { scribe(i). }
```

PYTHON (DESTINO)

```
i = 0 while i < x: print(i) i = i + 1
```

Mantém a equivalência semântica de inicialização, condição e incremento sem depender da função range().

Tabela de Equivalência: Latim → Python

Comando em Latim	Equivalente em Python	Descrição / Comportamento
<code>initium ... finis.</code>	Escopo Principal	Define as fronteiras obrigatórias do programa fonte
<code>numerus</code>	<code>int()</code>	Tipo inteiro; gera cast no <code>input()</code>
<code>decimalis</code>	<code>float()</code>	Tipo decimal (ponto flutuante)
<code>textus</code>	<code>str</code>	Tipo string para cadeias de caracteres
<code>logicum</code>	<code>bool</code>	Tipo booleano para expressões lógicas
<code>scribe(item).</code>	<code>print(item)</code>	Saída de dados no console
<code>lege(var).</code>	<code>input()</code>	Captura do teclado com conversão automática de tipo
<code>si (expr) { }</code>	<code>if expr:</code>	Condicional simples; parser elimina chaves e aplica recuo
<code>aliter { }</code>	<code>else:</code>	Condicional composta (ramificação senão)
<code>dum (expr) { }</code>	<code>while expr:</code>	Laço de repetição condicional contínua
Comando em Latim	Equivalente em Python	Descrição / Comportamento

Da Linguagem Fonte ao Python – Exemplo Real

CÓDIGO EM LATIM (FONTE)

```
initium numerus x. lege(x). si (x > 0) {  
  scribe(x). } aliter { scribe(0). }  
finis.
```



CÓDIGO PYTHON (DESTINO)

```
x = int(input()) if x > 0: print(x)  
else: print(0)
```

O transpilador **elimina as declarações de tipo**, aplica o **cast correto no input()**, **descarta as chaves** e gera a **identação automaticamente**.

Principais Desafios Enfrentados e Soluções Adotadas

Desafio	Solução Adotada
! Tipagem declarativa vs. dinâmica	✓ Tabela de Símbolos com consulta em tempo de transpilação
! Delimitadores { } vs. indentação	✓ Parser descarta chaves e gera recuo de 4 espaços
! Laço <code>pro</code> sem equivalente direto	✓ Geração de <code>while</code> com inicialização e incremento explícitos
! Segurança de tipos no <code>input()</code>	✓ Cast automático baseado no tipo registrado na Tabela de Símbolos
! Pontuação obrigatória <code>(.)</code>	✓ Lexer consome o ponto como delimitador de instrução sem emití-lo na saída

Conclusão – Equivalência Semântica Alcançada

✓ Conversão Sem Perda

O transpilador demonstra que é possível converter uma linguagem com **sintaxe rígida e tipagem declarativa** para Python **sem perda de comportamento**.

☰ Fundamentos Teóricos

O projeto reforça conceitos fundamentais de **Teoria da Computação**: autômatos, gramáticas formais, análise léxica/sintática/semântica e geração de código.

⚖️ Equivalência Garantida

A equivalência semântica foi garantida em todos os níveis: **tipos de dados, entrada/saída, estruturas condicionais e laços de repetição**.

🔍 Validação Imediata

A escolha do Python como destino permitiu **validação imediata e legível** do código gerado, cumprindo os requisitos do barema com clareza e rigor técnico.

Obrigado!

Dúvidas e considerações são bem-vindas.

